MLOps Exam Assignment

Weather Rain Prediction MLOps Pipeline

Group Name

Amisha Magar

Samad Ahmad

Nasrin Miya

1. Introduction

This project aims to develop a complete MLOps pipeline for weather prediction. The system predicts whether it will rain tomorrow based on historical weather data.
The focus of the project is not only model performance but also building a reproducible and deployable machine learning pipeline. The system includes data ingestion, preprocessing, model training, deployment via API, and monitoring.
The solution is designed to simulate a real-world production environment.

2. Pipeline Overview

The pipeline consists of the following components:

• Data Ingestion: Weather data is collected from the Open-Meteo API.

• Preprocessing: The data is cleaned and the target variable is created.

• Feature Engineering: Important features like maximum temperature, minimum temperature, and precipitation are used.

• Model Training: A Logistic Regression model is trained on the processed data.

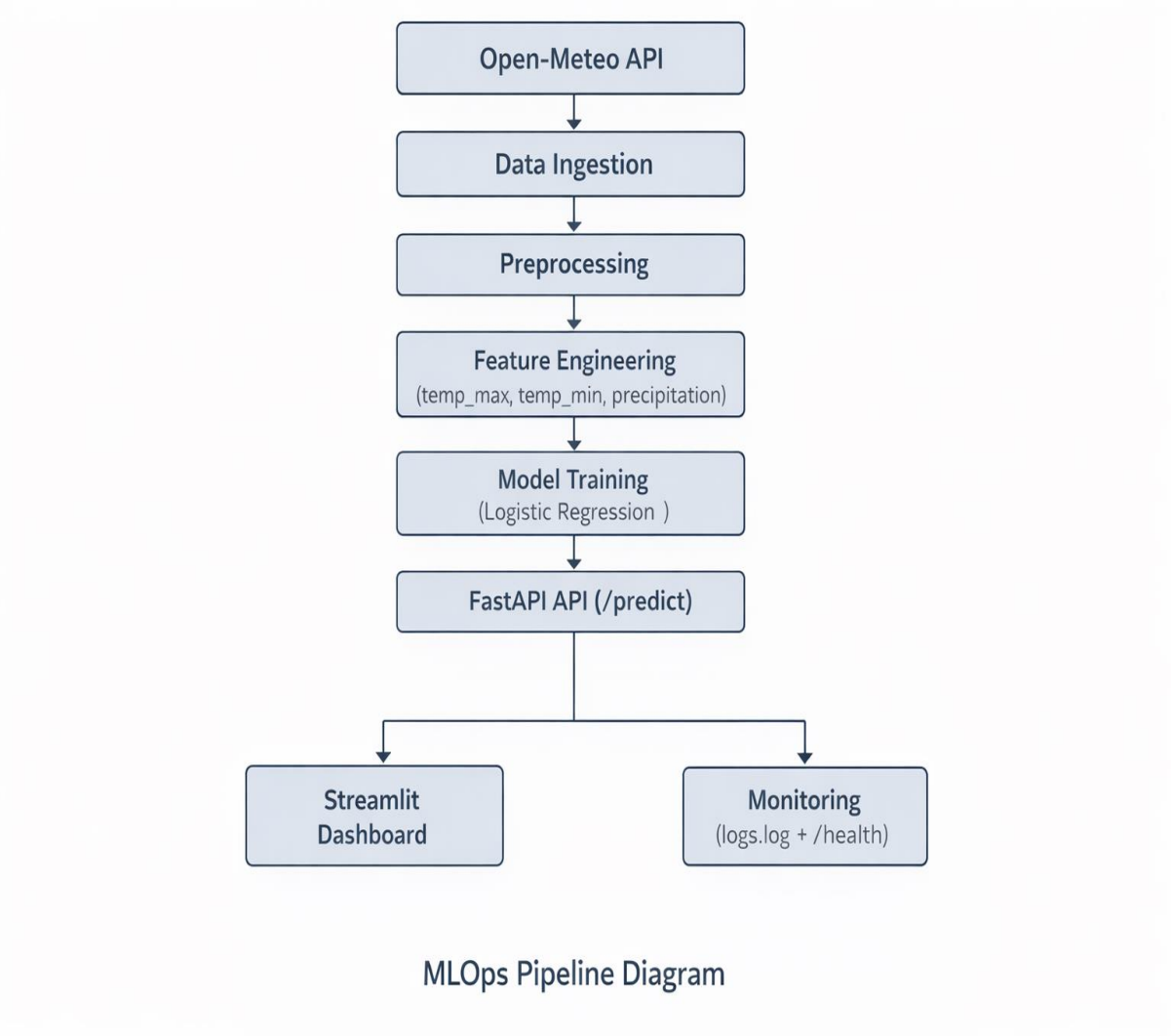
• Model Storage: The trained model is saved as a file for later use.

• Deployment: The model is deployed using FastAPI and can be used through the /predict endpoint.

• Frontend: A Streamlit dashboard is used so users can easily input data and get predictions.

• Monitoring: All prediction requests are saved in logs.log to track system usage.

• Containerization: The application is packaged using Docker to ensure it runs the same everywhere.



The diagram shows the flow of data from ingestion to deployment and monitoring.

3. Data Ingestion

The pipeline supports both original and new data ingestion. During the first execution, weather data is collected from the Open-Meteo API and stored as the initial dataset (original data). When the pipeline is executed again, it retrieves updated weather data (new data).

The ingestion process is automated using a Python script that requests data from the Open-Meteo API and stores it locally in CSV format. This ensures reproducibility and allows the pipeline to be executed repeatedly with updated data.

The raw data is stored in the `data/raw` directory.

4. Data Preprocessing

The raw data is processed to prepare it for model training. A new target variable called **rain_tomorrow** is created based on precipitation values.

The precipitation values are shifted by one day to represent whether it will rain the next day. The target is then converted into a binary classification problem, where 1 indicates rain and 0 indicates no rain.

Missing values created during shifting are removed to ensure clean data.

5. Feature Engineering

The model uses the following features:

- Maximum temperature
- Minimum temperature
- Precipitation

These features are selected based on their relevance to weather conditions and their influence on rainfall prediction.

The feature selection is kept simple to focus on building a robust and deployable MLOps pipeline.

6. Model Training

A Logistic Regression model is used for prediction. The dataset is split into training and testing sets using an 80/20 ratio.

The model is trained on the training data and evaluated on the test data. Performance is measured using accuracy, precision, recall, and F1-score.

The trained model is saved as a file for use during deployment.

Logistic Regression was chosen because it is simple, easy to understand, and suitable for binary classification. This allows the focus to be on building the MLOps pipeline rather than complex model tuning.

7. Results

The model was evaluated on the test dataset after training. The achieved accuracy was approximately 0.61.

The classification report shows that the model can predict both classes (rain and no rain), although performance is limited due to the relatively small dataset.

In addition to accuracy, the model was evaluated using precision, recall, and F1-score. These metrics provide a better understanding of model performance for both classes.

```
✅ Model trained successfully
Accuracy: 0.6111

Classification Report:

              precision    recall  f1-score   support

           0       0.62      0.80      0.70        10
           1       0.60      0.38      0.46         8

    accuracy                           0.61        18
   macro avg       0.61      0.59      0.58        18
weighted avg       0.61      0.61      0.59        18

✅ Model saved to models/rain_model.pkl
```

Example API prediction:

Input:

```
{
  "temp_max": 20,
  "temp_min": 10,
  "precipitation": 2
}
```

Output:

```
{
  "prediction": 0,
  "result": "No rain tomorrow"
}
```

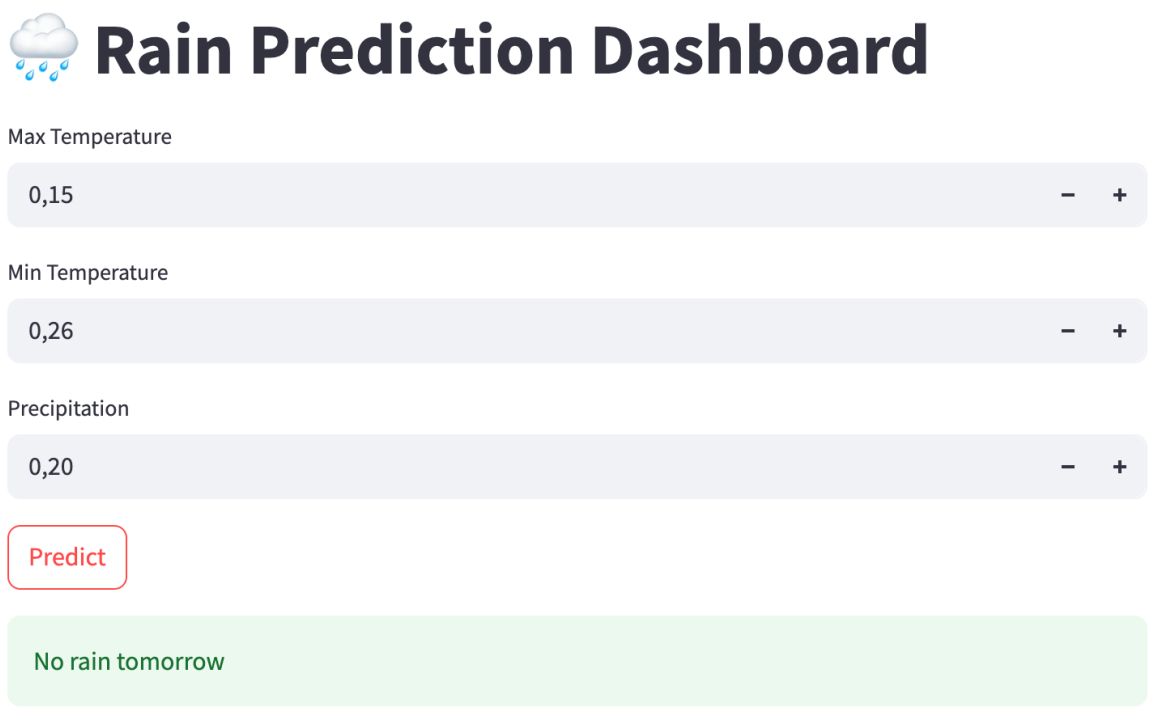
8. Deployment

The system is triggerable on demand through the FastAPI /predict endpoint. A user can send input data through an HTTP request and receive a prediction immediately. This satisfies the requirement that the pipeline should be runnable continuously or triggered when needed.

The API returns predictions in JSON format, making it easy to integrate with other systems.

Additionally, a Streamlit dashboard is implemented as a frontend, allowing users to input data and get predictions in a user-friendly way.

This setup simulates a real-world production environment where models are deployed as services and accessed by users or applications.



The following image shows the Streamlit dashboard used for real-time prediction.

9. Monitoring

Monitoring is implemented using logging in the FastAPI application. Each prediction request, including input values and prediction results, is stored in the logs.log file. This makes it possible to track model usage and inspect system behaviour over time.

In addition, the system includes a /health endpoint to verify that the API is running correctly.

10. Reproducibility and Artifact Management

The pipeline stores important artifacts in structured folders. Raw weather data is saved in `data/raw/weather.csv`, processed data is stored in the processed data folder, the trained model is stored in `models/rain_model.pkl`, and API prediction logs are stored in `logs.log`.

This structure ensures that the pipeline can be reproduced and executed consistently with the same results.

11. Containerization

The application is containerized using Docker to ensure consistent execution across different environments.

A Dockerfile is used to define the environment, install dependencies, and run the FastAPI server.

This makes the system portable and easy to deploy.

12. Conclusion

This project demonstrates the development of a complete MLOps pipeline for weather prediction.

The system integrates data ingestion, preprocessing, feature engineering, model training, deployment, monitoring, and a user-friendly frontend into a single workflow.

The focus on reproducibility, deployment, and usability ensures that the solution can be applied in real-world scenarios.